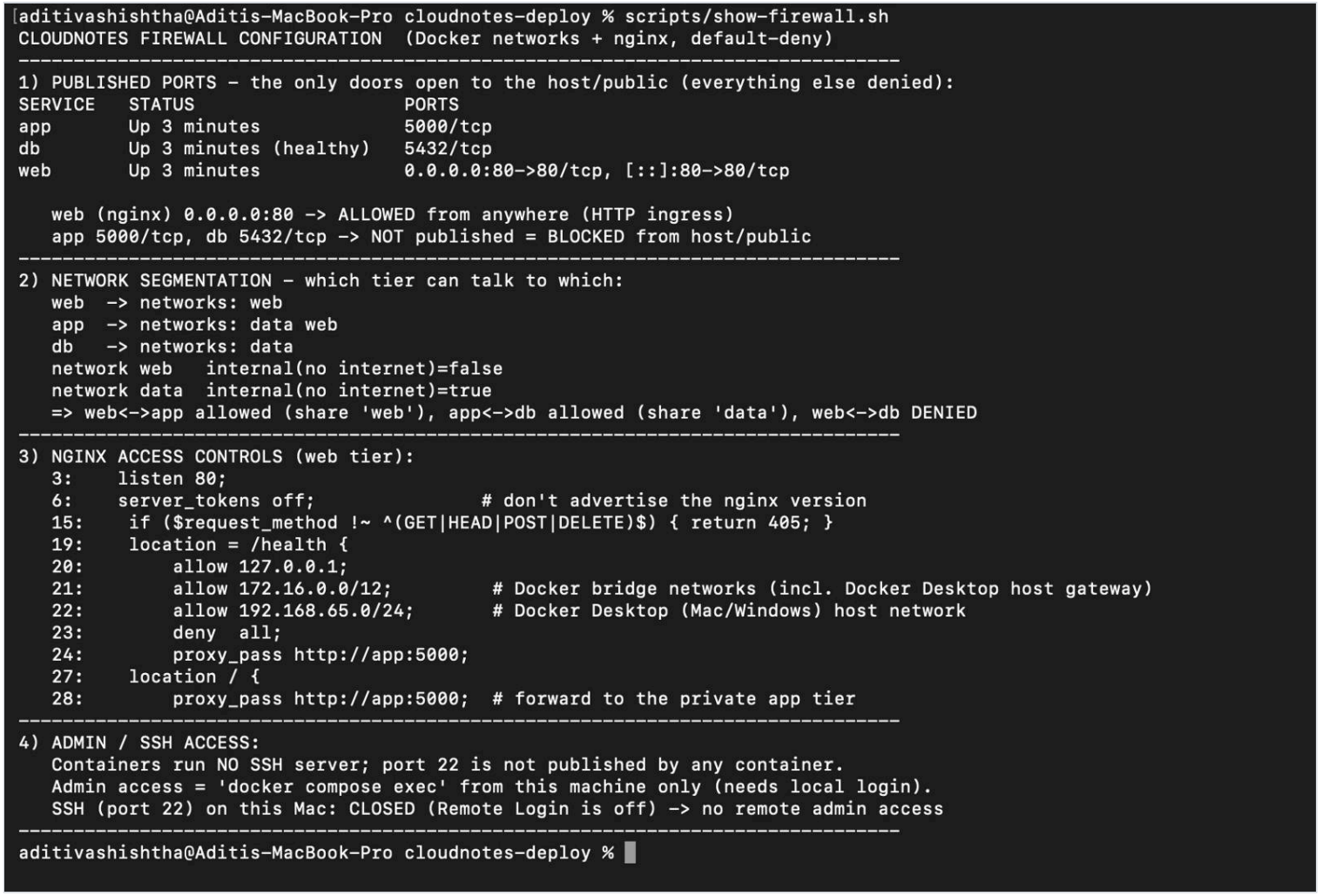
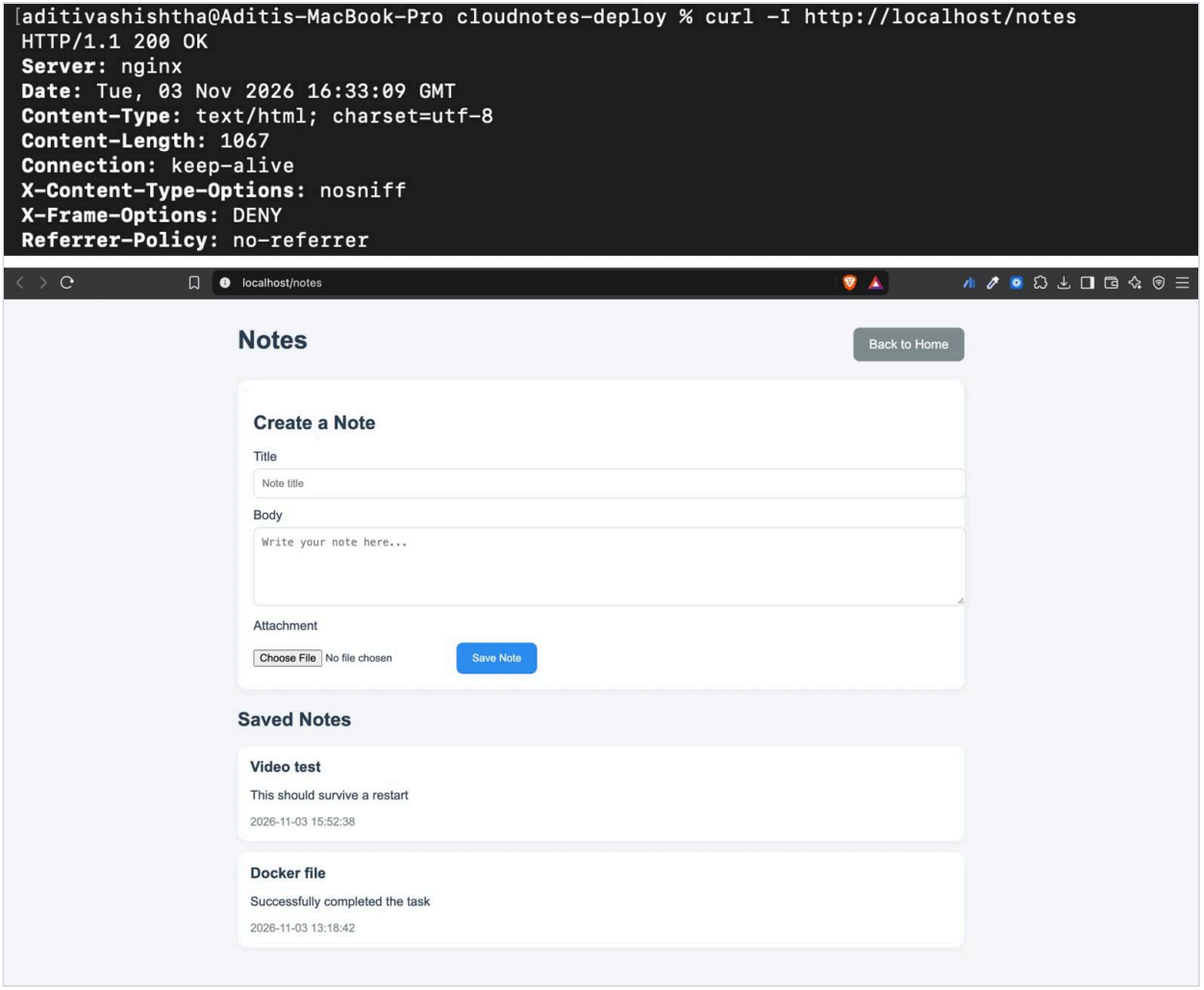
## Screenshot 1 — Firewall Configuration




**Caption:** Only the web tier (nginx) publishes a port: 0.0.0.0:80. The app (5000) and PostgreSQL (5432) publish nothing, so they are unreachable from outside (default-deny). Two Docker networks segment the tiers: 'web' (nginx + app) and 'data' (app + db, internal, no internet). nginx is the single entry point, allows only GET/HEAD/POST/DELETE and restricts the /health admin endpoint to private addresses. No container runs SSH, so admin access is local only. These rules protect the database and app from direct public access.

# Screenshot 2 — Allowed Traffic Verification



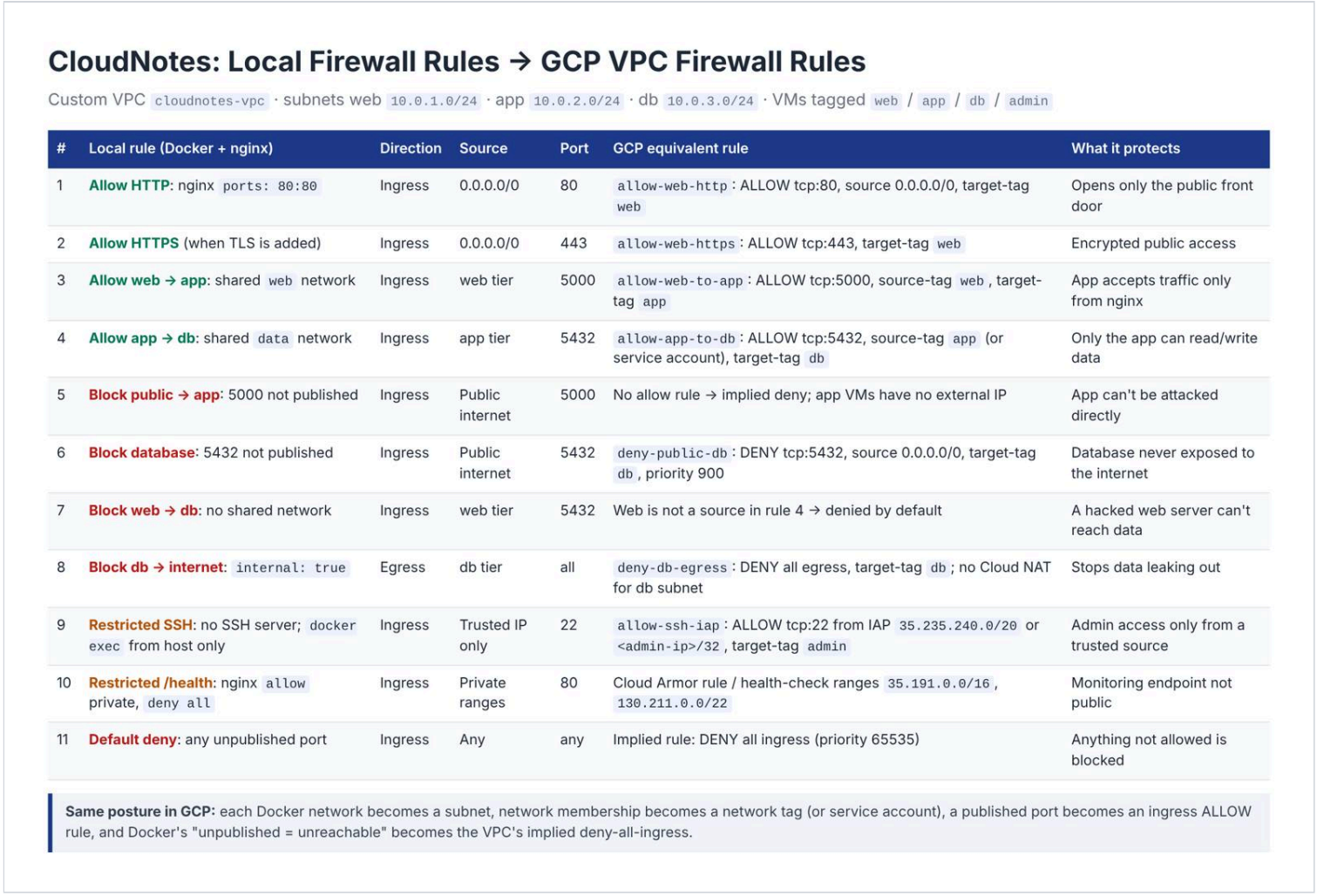

**Caption:** CloudNotes loads through nginx on http://localhost (port 80) with HTTP 200 OK. This traffic is allowed because port 80 on the web tier is the one intended public entry point; nginx then forwards the request privately to the app tier on port 5000, and only the app talks to PostgreSQL.

## Screenshot 3 — Blocked Traffic Verification

```
[aditivashishtha@Aditis-MacBook-Pro cloudnotes-deploy % nc -zv localhost 5432
 nc: connectx to localhost port 5432 (tcp) failed: Connection refused
 nc: connectx to localhost port 5432 (tcp) failed: Connection refused
[aditivashishtha@Aditis-MacBook-Pro cloudnotes-deploy % nc -zv localhost 5000
 nc: connectx to localhost port 5000 (tcp) failed: Connection refused
 nc: connectx to localhost port 5000 (tcp) failed: Connection refused
[aditivashishtha@Aditis-MacBook-Pro cloudnotes-deploy % docker compose exec web nc -zv db 5432
 nc: bad address 'db'
```

**Caption:** Connecting to PostgreSQL (5432) and directly to the app (5000) from the host is refused, and the web tier cannot even resolve the database ('bad address db'). These requests are blocked because those ports are never published and the database sits on an internal network shared only with the app tier — the principle of least privilege.

# Screenshot 4 — GCP Firewall Mapping Table



## CloudNotes: Local Firewall Rules → GCP VPC Firewall Rules

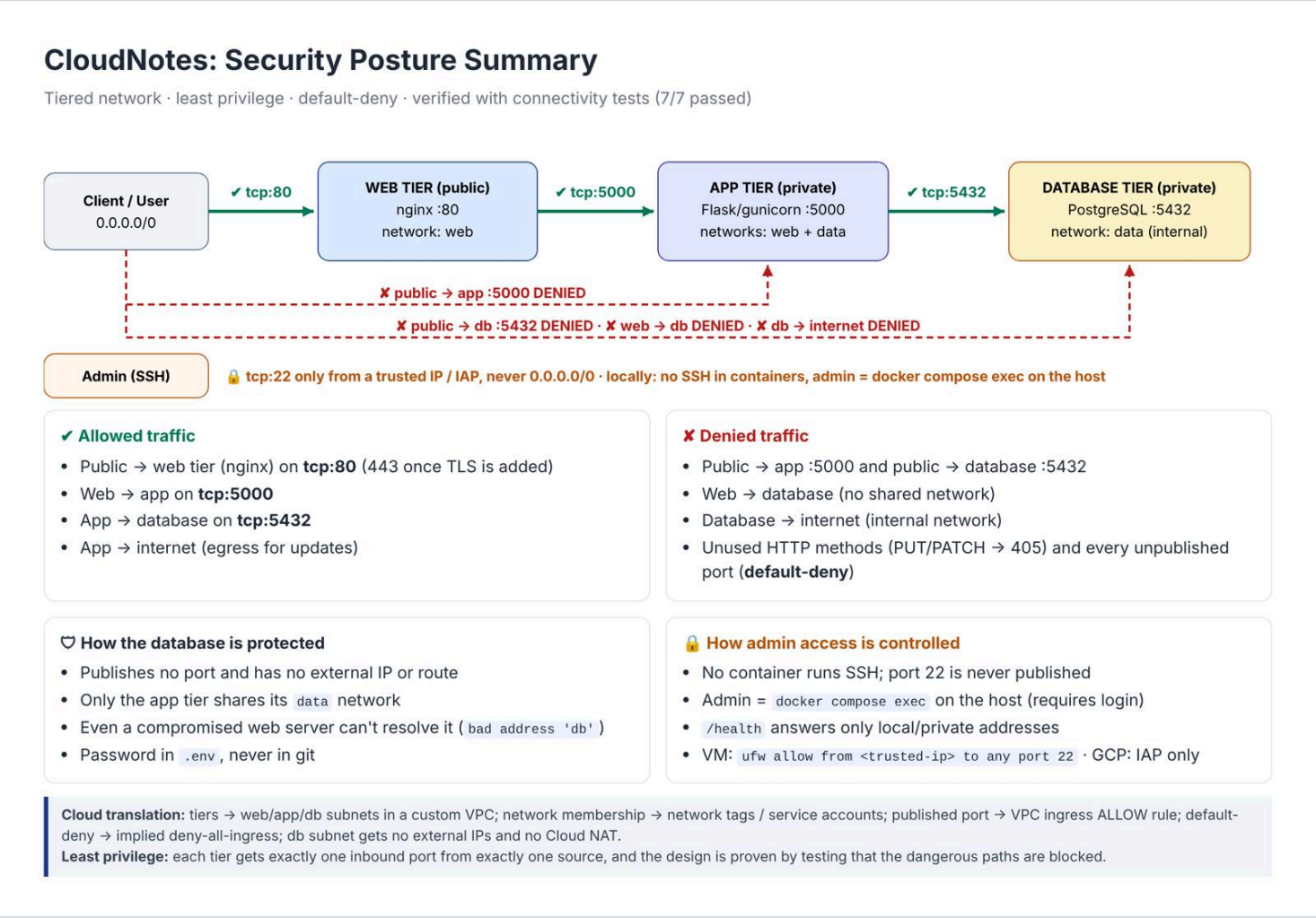
Custom VPC `cloudnotes-vpc` · subnets web `10.0.1.0/24` · app `10.0.2.0/24` · db `10.0.3.0/24` · VMs tagged `web` / `app` / `db` / `admin`

| # | Local rule (Docker + nginx) | Direction | Source | Port | GCP equivalent rule | What it protects |
|---|---|---|---|---|---|---|
| 1 | **Allow HTTP**: nginx `ports: 80:80` | Ingress | 0.0.0.0/0 | 80 | `allow-web-http` : ALLOW tcp:80, source 0.0.0.0/0, target-tag `web` | Opens only the public front door |
| 2 | **Allow HTTPS** (when TLS is added) | Ingress | 0.0.0.0/0 | 443 | `allow-web-https` : ALLOW tcp:443, target-tag `web` | Encrypted public access |
| 3 | **Allow web → app**: shared `web` network | Ingress | web tier | 5000 | `allow-web-to-app` : ALLOW tcp:5000, source-tag `web` , target-tag `app` | App accepts traffic only from nginx |
| 4 | **Allow app → db**: shared `data` network | Ingress | app tier | 5432 | `allow-app-to-db` : ALLOW tcp:5432, source-tag `app` (or service account), target-tag `db` | Only the app can read/write data |
| 5 | **Block public → app**: 5000 not published | Ingress | Public internet | 5000 | No allow rule → implied deny; app VMs have no external IP | App can't be attacked directly |
| 6 | **Block database**: 5432 not published | Ingress | Public internet | 5432 | `deny-public-db` : DENY tcp:5432, source 0.0.0.0/0, target-tag `db` , priority 900 | Database never exposed to the internet |
| 7 | **Block web → db**: no shared network | Ingress | web tier | 5432 | Web is not a source in rule 4 → denied by default | A hacked web server can't reach data |
| 8 | **Block db → internet**: `internal: true` | Egress | db tier | all | `deny-db-egress` : DENY all egress, target-tag `db` ; no Cloud NAT for db subnet | Stops data leaking out |
| 9 | **Restricted SSH**: no SSH server; `docker exec` from host only | Ingress | Trusted IP only | 22 | `allow-ssh-iap` : ALLOW tcp:22 from IAP `35.235.240.0/20` or `<admin-ip>/32` , target-tag `admin` | Admin access only from a trusted source |
| 10 | **Restricted /health**: nginx `allow` private, `deny all` | Ingress | Private ranges | 80 | Cloud Armor rule / health-check ranges `35.191.0.0/16` , `130.211.0.0/22` | Monitoring endpoint not public |
| 11 | **Default deny**: any unpublished port | Ingress | Any | any | Implied rule: DENY all ingress (priority 65535) | Anything not allowed is blocked |

> **Same posture in GCP:** each Docker network becomes a subnet, network membership becomes a network tag (or service account), a published port becomes an ingress ALLOW rule, and Docker's "unpublished = unreachable" becomes the VPC's implied deny-all-ingress.

**Caption:** In GCP the same posture becomes a custom VPC with web/app/db subnets. Docker network membership maps to network tags (or service accounts): allow tcp:80/443 from 0.0.0.0/0 to 'web', tcp:5000 from 'web' to 'app', tcp:5432 from 'app' to 'db', SSH only from IAP/trusted IP to 'admin', an explicit deny for public→db, deny db egress, and the implied deny-all-ingress for everything else.

## Screenshot 5 — Security Posture Summary




**Caption:** The design follows least privilege: each tier accepts exactly one port from exactly one source, everything else is denied by default, the database has no public port and no internet route, and admin access is limited to a trusted source. The posture is proven by connectivity tests, not assumed.